ETC3250/5250 Introduction to Machine Learning

Week 8: Support vector machines, nearest neighbours and regularisation

Jack Jewson

Jack.Jewson@monash.edu

Department of Econometrics and Business Statistics

ETC3250/5250 Lecture 8

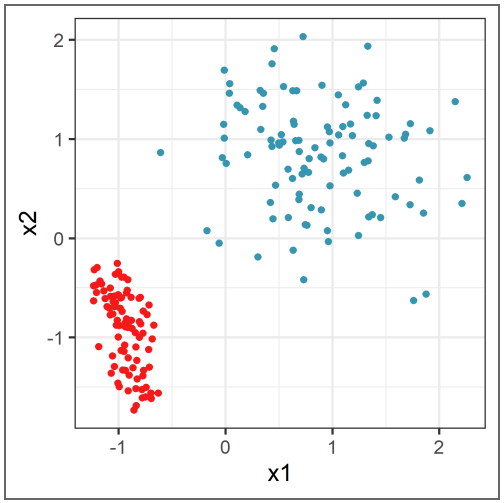
Overview

We will cover:

- Separating hyperplanes
- Non-linear kernels
- Really simple models using nearest neighbours
- Regularisation methods

ETC3250/5250 Lecture 8

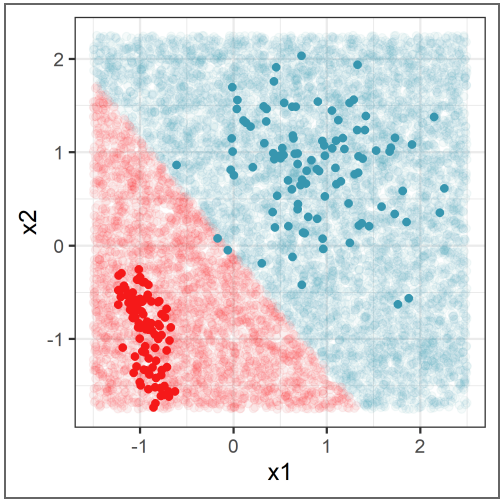
Why use support vector machines?



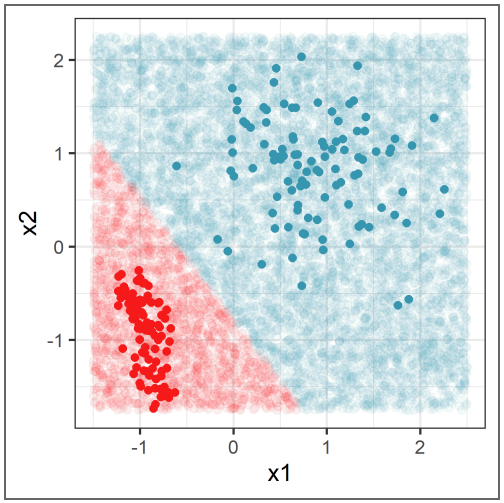
Where would you put the boundary to classify these two groups?

Here's where LDA puts the boundary.

ETC3250/5250 Lecture 8



What's wrong with it?



Why is this the better fit?

ETC3250/5250 Lecture 8

Separating hyperplanes (1/4)

LDA is an example of a classifier that generates a separating hyperplane

```
lda_fit$fit$scaling
```

```
    LD1
x1 -1.9
x2 -1.6
```

As the data was standadrised, a new observation is class blue class if $-1.9 \times x1 - 1.6 \times x2 > 0$
Which is the same as $x2 > -\frac{1.9}{1.6} \times x1$

ETC3250/5250 Lecture 8

Separating hyperplanes (2/4)

Separating hyperplane produced by LDA on 4D penguins data, for Gentoo vs Adelie. (It is 3D.)

Separating hyperplanes (2/4)

Separating hyperplane produced by LDA on 4D penguins data, for Gentoo vs Adelie. (It is 3D.)

ETC3250/5250 Lecture 8

Separating hyperplanes (3/4)

The equation of p-dimensional hyperplane is given by
$\beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p = 0$
Given a hyperplane we can define $g(x) = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p$ and define as classifier as predicting to sign of $g(x)$.
LDA estimates β_j based on the sample statistics, means for each class and pooled covariance.
SVM estimates β_j based on support vectors (◦, ◦), observations on the border between the two groups. Thus, the boundary will divide in the gap.

ETC3250/5250 Lecture 8

Separating hyperplanes (4/4)

LDA

$$x\, S^{-1}(\bar{x}_A - \bar{x}_B) - \frac{\bar{x}_A + \bar{x}_B}{2} S^{-1}(\bar{x}_A - \bar{x}_B) = 0$$

resulting in:

$$\hat{\beta}_0 = -\frac{\bar{x}_A + \bar{x}_B}{2} S^{-1}(\bar{x}_A - \bar{x}_B)$$

$$\begin{pmatrix} \hat{\beta}_1 \\ \hat{\beta}_2 \\ \vdots \\ \hat{\beta}_p \end{pmatrix} = S^{-1}(\bar{x}_A - \bar{x}_B)$$

SVM

Set $y_A = 1, y_B = -1$, and x_j standarised, s = number of support vectors.

ETC3250/5250 Lecture 8

$$\begin{pmatrix} \hat{\beta}_1 \\ \hat{\beta}_2 \\ \vdots \\ \hat{\beta}_p \end{pmatrix} = \sum_{k=1}^{s} (\alpha_k y_k) x_{kj}$$

ETC3250/5250 Lecture 8

Linear support vector machine classifier (1/4)

Assume the data is linearly seperable

Many possible separating hyperplanes, which is best?

ETC3250/5250 Lecture 8

Linear support vector machine classifier (2/4)

The best separating hyperplane has the biggest margin

Can we write this mathematically?

ETC3250/5250 Lecture 8

Linear support vector machine classifier (3/4)

As a result we can find the maximum margin separating hyper plane by

Minimise wrt $\beta_j, j = 0, \ldots, p$

$$\frac{1}{2}\sqrt{\sum_{j=1}^{p} \beta_j^2}$$

subject to $y_i(\sum_{j=1}^{p} x_{ij}\beta_j + \beta_0) \geq 1.$

ETC3250/5250 Lecture 8

Linear support vector machine classifier (4/4)

Classify the test observation x based on the sign of
$g(x) = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p$

- If $g(x) > 0$, class 1, and if $g(x) < 0$, class -1, i.e. $\hat{y} = \text{sign}(g(x))$.
- $g(x)$ far from zero $\Rightarrow x$ lies far from the hyperplane + **more confident** about our classification - i.e. we can turn this into a probability

ETC3250/5250 Lecture 8

Tidymodels Implementation (1/2)

https://parsnip.tidymodels.org/reference/svm_linear.html

```
svm_spec <- svm_linear() |>
  set_mode("classification") |>
  set_engine("kernlab")
```

```
svm_fit <- svm_spec |>
  fit(cl ~ ., data = pebbles)
```

```
Setting default kernel parameters
```

```
pebbles_grid <- pebbles_grid |>
  mutate(cl_svm = factor(predict(svm_fit$fit,
                                 pebbles_grid)))
```

https://emilhvitfeldt.github.io/ISLR-tidymodels-labs/09-support-vector-machines.html

ETC3250/5250 Lecture 8

Tidymodels Implementation (2/2)

```
library(kernlab)
svm_fit |>
  extract_fit_engine() |>
  plot(data=pebbles)
```

ETC3250/5250 Lecture 8

Non-seprable data

However real data is not separable…

… unless we are allowed to move observations

ETC3250/5250 Lecture 8

Soft threshold, when no separation

ETC3250/5250 Lecture 8

Non-seprable SVM

Introduce an amount ξ_i by which we can move any observation

As a result we can find the maximum margin minimum movement separating hyper plane by

Minimise wrt $\beta_j, j = 0, \ldots, p$ and $\xi_i, i = 1, \ldots, n$

$$\frac{1}{2}\sqrt{\sum_{j=1}^{p} \beta_j^2} + C \sum_{i=1}^{n} \xi_i$$

subject to $y_i(\sum_{j=1}^{p} x_{ij}\beta_j + \beta_0) \geq 1 - \xi_i$ and $\xi_i \geq 0$.

where C is a regularisation parameter that controls the trade-off between maximizing the margin and minimizing the misclassifications $\sum_{i=1}^{n} \xi_i$.

ETC3250/5250 Lecture 8

The cost parameter

```
svm_spec1 <- svm_linear(cost = 0.01) |>
  set_mode("classification") |>
  set_engine("kernlab")
```

```
Setting default kernel parameters
```

```
svm_spec2 <- svm_linear(cost = 100) |>
  set_mode("classification") |>
  set_engine("kernlab")
```

```
Setting default kernel parameters
```

You can use cross-validation to set the cost

Using kernels for non-linear classification

Note: In Linear SVM,

$$\hat{\beta} = \sum_{k=1}^{s} (\alpha_k y_k) x_{kj}$$

and therefore for a new observation x

$$g(x) = \hat{\beta_0} + \hat{\beta}^T x.$$

$$= \hat{\beta_0} + \sum_{k \in \square} \alpha_k y_k \langle x_k, x \rangle.$$

where the inner product is defined as

$$\langle x_1, x_2 \rangle = x_{11} x_{21} + x_{12} x_{22} + \cdots + x_{1p} x_{2p}$$

$$= \sum_{j=1}^{p} x_{1j} x_{2j}$$

The data might not be linear separable in the original space, but they are in a higher dimensional space

Source: Grace Zhang @zxr.nju

ETC3250/5250 Lecture 8

The kernel trick allows us to efficiently fit a linear hyper plane in a high dimensional space

ETC3250/5250 Lecture 8

Using kernels for non-linear classification

Source: Grace Zhang @zxr.nju

A kernel function is an inner product of vectors mapped to a (higher dimensional) feature space.

$\square(x_1, x_2) = \langle \psi(x_1), \psi(x_2) \rangle$

We can generalise SVM to a non-linear classifier by replacing the inner product with the kernel function as follows:

$$g(x) = \beta_0 + \sum_{k \in \square} \alpha_k y_k \langle \psi(x_k), \psi(x) \rangle$$

$$= \beta_0 + \sum_{k \in \square} \alpha_i \square(x_k, x).$$

Common kernels: radial basis function (RBF)

$$\square(x_k, x) = \exp\left(-\frac{(x_k - x)^2}{2\sigma^2} \right)$$

ETC3250/5250 Lecture 8

Non-linear SVM in practice

---

https://parsnip.tidymodels.org/reference/svm_rbf.htm

```
svm_rbf_spec <- svm_rbf(cost = 1, rbf_sigma = NULL) |> #
  set_mode("classification") |>
  set_engine("kernlab")
```

```
svm_rbf_fit <- svm_rbf_spec |>
  fit(y ~ ., data = sim_data2)
```

As well as grid searching over the `cost` parameter, you can also grid search over the `rbf_sigma`

ETC3250/5250 Lecture 8

Non-linear SVM in practice

ETC3250/5250 Lecture 8

Learned Feature Engineering

- Remember how we described the hidden layer outputs in a single hidden layer as learned feature encodings
  - The outputs of the hidden-layer is a non-linear function applied to a weighted sum of the features
  - The final layer was just logistic regression on the outputs of the hidden layer
- Kernel SVM uses the same idea
  - We fit a linear separating hyperplane in a high dimensional space defined by the $\psi(x)$
  - Using the kernel trick to evaluate $\square(x_1, x_2) = \langle \psi(x_1), \psi(x_2) \rangle$ allows us to do this efficently

ETC3250/5250 Lecture 8

- And using cross-validation to learn the kernel hyperparameter is learning which high-dimensional space (i.e. which $\psi(\cdot)$) best separates the data

ETC3250/5250 Lecture 8

Really simple models

ETC3250/5250 Lecture 8

k-nearest neighbours

k-nearest neighbours (kNN) is possibly the simplest non-linear non-parametric supervised learning model

Idea: For any new observation x with unknown response (class or value), predict its response based on the average responses $\{y_i\}_{i=1}^n$ of training set observations whose features $\{x_i\}_{i=1}^n$ are closest to x

More like a recipe for prediction than a model

All you need to specify is

- A distance $d(x, x_k)$
- A number of neighbours K

ETC3250/5250 Lecture 8

Recipe for prediction

- Specify a distance metric $d(\cdot, \cdot)$ (usually defaults to $d(x_1, x_2) = \sqrt{\sum_{j=1}^{p} (x_{1j} - x_{2j})^2}$) and a number of neighbours K
- When predicting new observation x
  - for $i = 1, \ldots, n$ evaluate $d(x, x_i)$
  - select the K observations with the smallest $d(x, x_i)$
  - Predict the response as the average of the y_i of the K nearest neighbours
    - In regression predict $\frac{1}{K} \sum_{k=1}^{K} y_{n_k}$
    - In Classification predict $P(y = C) = \frac{1}{K} \sum_{k=1}^{K} \mathbb{I}(y_{n_k} = C)$

ETC3250/5250 Lecture 8

k-nearest neighbours

Predict y using the $k-$ nearest neighbours from observation of interest.

ETC3250/5250 Lecture 8

Details

- Use cross-validation to learn the K that best predicts out of sample data
- No good at looking at in-sample predictions as each observation is its own neighbour
- Can also use cross-validation to learn the distance metric e.g. using $d(x_1, x_2) = \sum_{j=1}^{p} |x_{1j} - x_{2j}|$
- Important to standardise the features so that each has an equal weight on the distance $d(\cdot, \cdot)$
- Rather than just returning the average of the response of the neighbours, it is often preferable to use a weighted average
  - Set $w_{n_k}(x) \propto 1/d(x, x_{n_k})$
  - In regression predict $\frac{1}{K} \sum_{k=1}^{K} w_{n_k}(x) y_{n_k}$
  - In classification predict $P(y = C) = \frac{1}{K} \sum_{k=1}^{K} w_{n_k}(x) \mathbb{I}(y_{n_k} = C)$

ETC3250/5250 Lecture 8

Implementation

https://parsnip.tidymodels.org/reference/nearest_neighbor.html

```
knn_spec <- nearest_neighbor(neighbors = 5, weight_func =
  set_mode("classification") |>
  set_engine("kknn")
```

```
knn_fit <- knn_spec |>
  fit(y ~ ., data = sim_data2)
```

```
sim_data2_grid <- tibble(
  x1 = runif(10000, -5, 5),
  x2 = runif(10000, -5, 5))


sim_data2_grid <- sim_data2_grid |>
  mutate(cl_knn = factor(predict(knn_fit$fit,
                                 sim_data2_grid)))
```

ETC3250/5250 Lecture 8

ETC3250/5250 Lecture 8

Implementation

ETC3250/5250 Lecture 8

Pros and Cons

Pros

- Very simple,
- Very few hyper parameters to tune
- Good at picking up local structure in your data

Cons

- Standardising the data gives all features the same weight,
- There is no automatic way to give important features increased weight or learn feature importances
- Fails in high dimensions because data is too sparse - in these cases the model needs some structure (assumptions)

Regularisation

ETC3250/5250 Lecture 8

What is the problem in high-high-D? (1/3)

20 observations and 2 classes: A, B. One variable with separation, 99 noise variables

---

What will be the optimal Logistic Regression coefficients?

What is the problem in high-high-D? (2/3)

Fit logistic regression with the first two variables.

```
parsnip model object


Call:  stats::glm(formula = cl ~ ., family =
stats::binomial, data = data)

Coefficients:
(Intercept)           x1           x2
       3.12       -33.67         2.95

Degrees of Freedom: 19 Total (i.e. Null);  17 Residual
Null Deviance:      28
Residual Deviance: 5.5e-10  AIC: 6
```

Coefficient for x1 MUCH higher than x2. As expected!

Predict the training and test sets

A B

ETC3250/5250 Lecture 8

```
A 10  0
B  0 10
```

```
  A B
A 5 0
B 0 5
```

Perfect!

ETC3250/5250 Lecture 8

What is the problem in high-high-D? (3/3)

What happens to the estimated coefficients as dimensions of noise increase?

Remember, the noise variables should have coefficient = ZERO.

ETC3250/5250 Lecture 8

How do you check? (1/2)

```
w <- matrix(runif(48*40), ncol=40) |>
  as.data.frame() |>
  mutate(cl = factor(rep(c("A", "B", "C", "D"), rep(12, 4))))
```

$n = 48, p = 40$ Class labels are randomly generated

```
logistic_mod <- multinom_reg(penalty = 0) |>
  set_engine("nnet") |>
  set_mode("classification") |>
  translate()


w_log_reg <-
  logistic_mod |>
  fit(cl~.,
      data = w)
```

Permutation is your friend, for high-dimensional data analysis.
Permute the class labels. and refit the model

```
set.seed(951)
ws <- w |>
  mutate(cl = sample(cl))
```

ETC3250/5250 Lecture 8

ETC3250/5250 Lecture 8

How do you check? (2/2)

- Permuting response, repeating the analysis, then make model summaries and diagnostic plots
- Comparing with test set is critical.
- If results (error/accuracy, low-d visual summary) on test set are very different than training, it could be due to high-dimensionality/overfitting.

How can you correct?

- Subset selection: reduce the number of variables before attempting to model
- Bagging: fit seperate models to different subsets of the data and then average these models
- Penalisation: change the optimisation criteria to include another term which makes it worse when there are more coefficients

Regularisation

Recall that in Logistic regression we set

$$\hat{\beta_0}, \hat{\beta_1} := \arg\max_{\beta_0 \in \mathbb{R}, \beta_1 \in \mathbb{R}^p} \log l_n(\beta_0, \beta_1)$$

where

$$l_n(\beta_0, \beta_1) := \prod_{i=1}^{n} P(Y_i = y_i | x, \beta_0, \beta_1)$$

Estimates the parameters of the model to make the data as likely as possible

Any change to a β_j that makes the data even a little more likely is preferable

Penalised logistic regressions sets

$$\hat{\beta_0}, \hat{\beta_1} := \arg\max_{\beta_0 \in \mathbb{R}, \beta_1 \in \mathbb{R}^p} \log l_n(\beta_0, \beta_1) - \frac{\lambda}{2} \sum_{j=1} \beta_j^2$$

If increasing a β_j only make the data a little more likely then this is no longer preferable

Set λ by cross-validation

https://parsnip.tidymodels.org/reference/logistic_reg.html

```r
logistic_mod <- logistic_reg(penalty = 0.1) |>
set_engine("glmnet") |>
set_mode("classification")
```

ETC3250/5250 Lecture 8

Regularisation

What happens to the estimated coefficients as dimensions of noise increase?

Remember, the noise variables should have coefficient = ZERO.

ETC3250/5250 Lecture 8

Next: K-means and hierarchical clustering

ETC3250/5250 Lecture 8